

CS6600 – Computer Architecture

Assignment 3: TAGE-SC-L Branch Prediction and Speculative Execution Report

Team Number 6

October 3, 2026

1 Team Members and Contributions

Name	Roll No.	Contribution
K V Veerendra Babu	EE25M102	TAGE core, core.cc
Anjaneya J S	EE25M011	Loop Predictor, speculation.cc
C V Tirumala Rao	EE25M067	Statistical Corrector, predictor_one.cc

Speculation controller, recovery logic, testing and final integration were done jointly. Individual contributions are identifiable through the Git history of the repository.

2 Design: Variant, Organisation, Parameters and Pipeline Integration

2.1 Selected variant

We implement a **scaled-down TAGE-SC-L following Seznec’s CBP-2014 design** (with the original TAGE as the core). The statistical corrector is a *reduced Multi-GEHL* corrector (CBP-2014, Sec. 5.2), and the loop predictor is a simplified version of the CBP-2014 loop predictor. All three components are in `PredictorOne` (slot 0) and all participate in the final prediction. Slots 1 and 2 are untouched.

Prediction flow.

1. **BTB lookup** (512-entry, direct-indexed, full-PC tag). A miss means “not a branch”: predict $PC + 4$. An entry whose taken target has not been learned yet (`target_valid=0`) can only predict not-taken.
2. On a BTB hit, J is always taken to the stored target. For BEQ/BNE, the **TAGE** direction is computed.
3. The **statistical corrector** (SC) may *invert* TAGE only if TAGE is low-confidence and the SC sum disagrees.
4. A **confident loop predictor** entry (confidence ≥ 2) has the final say. A wrong confident loop prediction resets that entry’s confidence to 0 immediately.
5. Target = BTB target if final direction is taken, else $PC + 4$.

2.2 Predictor organisation and parameters

Every table is **direct-indexed**: a hash selects one entry and its stored tag is compared. No associative search is used anywhere.

Table 1: TAGE tagged tables (3-bit counter, 2-bit useful, 1 valid bit per entry).

Table	History length	Entries	Index bits	Tag bits
T0	4	256	8	7
T1	9	256	8	7
T2	13	256	8	7
T3	24	256	8	8
T4	37	128	7	9
T5	53	256	8	10
T6	91	128	7	10
T7	145	64	6	11
T8	226	64	6	13
T9	359	64	6	13

Table 2: Summary of all predictor components.

Component	Parameters
BTB	512 entries, direct-mapped by $PC[10 : 2]$, stores 32-bit PC tag, 32-bit target, valid, is-conditional, target-valid. Allocated/updated when a branch resolves; a new owner of an entry discards the previous branch’s target.
TAGE base	2048 entries of 2-bit counters, indexed by $PC[12 : 2]$.
TAGE tagged	10 tables (above), geometric history 4–359. Index = $PC \oplus$ folded GHR $\oplus$ rotated path history; tag = $PC \oplus$ two folds of GHR.
Provider / alternate	Longest matching table = provider; next-longest match (else base) = alternate. <code>use_alt_on_na</code> : 32×4 -bit counters, indexed by PC; if provider is weak (counter 3 or 4) and counter ≥ 8 , the alternate prediction is used.
Allocation	On a TAGE misprediction, allocate up to 2 entries in longer tables with $u = 0$ (non-consecutive tables), new counter weak towards actual outcome. If none free: decrement u in candidates and increment 8-bit <code>tick</code> ; when <code>tick</code> wraps, all u bits are decremented (usefulness ageing).
Usefulness	u (2 bit) incremented when provider is right and alternate is wrong.
SC	6 direct-indexed tables: 2 bias tables (128 entries each) + 4 GEHL tables (history 4, 16, 64, 256; sizes 64, 64, 128, 128). Each entry: 6-bit signed counter + 7-bit tag. Sum of matching counters plus $\pm 2 \cdot 6$ TAGE bias gives score; sign of score = SC direction. Invert TAGE only if TAGE is low-confidence, the SC sign disagrees and $ score \geq 4$. Train (fixed threshold 8) on SC misprediction or $ score < 8$.
Loop	16 entries, direct-mapped: $PC[5 : 2] =$ index, $PC[31 : 6] =$ 26-bit tag. Fields: 16-bit current iteration, 16-bit trip count, 2-bit confidence, continue-direction, valid. Predicts the continue direction until the trip count, then the exit direction.
History	359-bit global history register (GHR), 27-bit path history (PHR).

2.3 History update and recovery policy

- **History is updated only at resolution** (inside `Update`), using the real outcome. Conditional branches shift in 1 outcome bit; unconditional jumps shift in 4 target bits; the PHR shifts in

$PC[2]$.

- Because history is never modified speculatively, **no history rollback is needed** on a squash – the wrong path never touches it.
- At **Predict**, a snapshot of the GHR/PHR is saved in a 256-slot ring (slot id carried in the prediction’s 47-bit **information** field). At **Update**, TAGE allocation recomputes indices/tags from this *same* snapshot, so training is consistent with the lookup. SC lookup snapshots (indices, tags, score) are indexed by the same slot id.
- Metadata of squashed instructions is simply never consumed; its ring slot is overwritten circularly (256 slots exceed the maximum number of in-flight predictions).

2.4 Pipeline integration

Fetch. `Core::Fetch` calls `BranchPredictionSystem::Predict` only after fetch-queue and I-cache link capacity checks pass, then issues `TryIssuePredictedFetch`. Next PC is `BranchPredictionSet::next_pc` (slot 0). The prediction set travels with the instruction through IF/ID and into the execution operation and checkpoint; it is never recomputed. Fetch stalls when the controller holds 128 checkpoints (`CanFetch`).

Decode. `ObserveInstruction` is called **exactly once per instruction** (the core remembers the last observed sequence number, so an instruction stalled in IF/ID is not re-observed). If the selected predictor flagged a non-branch as a branch, next PC is set to $PC + 4$, `Machine::SquashAfter` discards the younger fetches (ready ones dropped, in-flight ones marked stale), the discarded fetches are recorded with `RecordSquash`, and a recovery cycle is counted. The false-branch instruction itself continues normally. For each control instruction (not HALT) a `BranchCheckpoint` (sequence, predicted next PC, prediction set) is added at dispatch.

Speculative dispatch. `SpeculationController::CanDispatch`: while any checkpoint is outstanding, only ADD/ADDI/SUB/AND/OR/XOR may dispatch. LW, SW, BEQ, BNE, J, HALT (and NOP) wait. Dispatch stays in program order; a blocked instruction blocks everything behind it. Younger ALU results sit in completion storage until in-order retirement.

Resolution. In `Core::Memory` the CONTROL unit’s output builds a `BranchResult` (actual direction and target). Two separate calls follow:

1. `BranchPredictionSystem::Resolve` – trains all predictors and updates branch/predictor metrics only.
2. `SpeculationController::Resolve` – compares the checkpoint’s stored `predicted_next_pc` with the actual next PC, **releases the branch’s own checkpoint**, and returns the authoritative `RecoveryDecision`.

Only the second call’s decision triggers a squash.

Squash. On misprediction: next PC $\leftarrow$ correct PC; `Machine::SquashAfter(seq)` (discards younger fetch entries, marks in-flight I-fetches stale so late responses are dropped, removes younger completions and reservations); `ExecutionPipeline::SquashAfter(seq)` on all 7 pipelines; `RemoveYoungerCheckpoints(seq)`; invalidate younger IF/ID; `RecordSquash`. The branch and everything older are preserved. The AGU hand-off to the data cache is retried every cycle until accepted (`TryIssueData` can fail when the pending-data queue is full), so a load can never be stranded in the AGU. A per-cycle `squash_sequence_` guard stops Decode/Fetch in the same cycle from copying stale younger state, preserving the WB, MEM, EX, ID, IF order and the current/next-state discipline.

2.5 Statistics: counter definitions

Table 3: Counter definitions (what event is counted, and when).

Counter	Event definition
<code>predictor0.lookups</code>	Every Predict call (all lookups, including wrong-path fetches).
<code>branches_found / not_found</code>	BTB hit / BTB miss at fetch (all lookups). This is the branch-identification hit/miss counter.
<code>predicted_taken / not_taken</code>	Final direction returned by Predict (all lookups).
<code>correct_predictions</code>	Resolved branches with right direction and right target.
<code>direction_mispredictions</code>	Resolved branches where predicted-taken (BTB hit & taken) $\neq$ actual taken. A taken branch that missed in the BTB counts here.
<code>target_mispredictions</code>	Resolved, direction right, taken, but target wrong. Mutually exclusive with direction errors (no double count).
<code>false_branch_predictions</code>	Decode saw a non-branch that Predict marked as a branch.
<code>missed_branches</code>	Decode saw a real branch that Predict did not identify.
<code>updates</code>	Update calls (once per resolved branch).
<code>branch.branches_resolved</code>	Resolved branches (cond. + uncond.), counted once at Resolve .
<code>branch.redirects</code>	Direction errors + target errors + false-branch redirects.
<code>branch.recovery_cycles</code>	One per recovery event: a resolve-time squash (decided by <code>SpeculationController::Resolve</code>) or a decode-time false-branch squash, counted in the cycle the redirect is issued. The refill delay is not a separate counter; it appears in the total cycle count.
<code>speculative_instructions_fetched</code>	Each <i>accepted</i> predicted fetch; stalled attempts are not counted. Max speculation depth = largest checkpoint count seen at fetch.
<code>speculative_instructions_dispatched</code>	Each instruction dispatched while at least one older branch checkpoint is unresolved (only ALU operations can, by the dispatch rule). Counted once, at dispatch.
<code>branch.squashed_*</code>	Wrong-path items removed by a squash, one counter each for fetches (fetch queue and IF/ID), execution-pipeline operations, completions and data requests (the last must be 0).

Consistency identities we check: `branches_resolved = correct + direction + target`; `redirects = direction + target + false_branch_redirects`; `squashed_data_requests = 0`; the TAGE provider counters sum to the resolved conditional branches.

Derived metrics.

$$\text{IPC} = \frac{I_{\text{retired}}}{C_{\text{cycles}}}, \quad \text{MPKI} = 1000 \cdot \frac{M_{\text{direction}} + M_{\text{target}}}{I_{\text{retired}}}.$$

Component-level counters (own instrumentation). All are plain counters kept inside `PredictorOne`; nothing is printed and the starter’s `PrintStatistics` is unchanged. They are read through the predictor’s accessors (`TageStatistics()`, `ScStatistics()`, `LoopMetrics()`, `BtbStatistics()`). **TAGE:** `tage.provider.T0..T9/base/alt` and `_correct` (who supplied TAGE’s direction at resolve and whether it was right; `alt` = alternate used instead of a weak provider), `base/tagged/use_alt` reads and writes, allocations and allocation failures. **SC:** `sc.lookups`, `tag.hits`, `overrides`, `overrides_resolved/correct/wrong`, allocations, bias/GEHL reads and writes. **Loop:** `loop.lookups`, `selected`, `correct`, `table_reads`,

`table_writes`, `replacements`. **BTB:** `btb.table_reads`, `table_writes`. TAGE and SC provider/override counters are updated at resolve (resolved branches only); lookup counters count every prediction, including wrong-path ones.

3 Assumptions and Departures from the Papers / Starter

Item	Departure / assumption & justification	Effect
Overall size	Total ≈ 73 Kbit vs. the 256 Kbit CBP-2014 budget; shorter tables (64–256 entries). Keeps the design lightweight, as the grading rewards cost.	Lower capacity; more aliasing on large programs.
History update	Updated at resolve, not speculatively (see Sec. 2.3). Removes the need for history repair.	Branches fetched while older ones are unresolved see slightly stale history; some accuracy loss.
Folded history	Folds are recomputed from the 359-bit GHR in software each lookup instead of incremental circular folded registers.	Simulation convenience only; hardware would add folded-history registers (not counted in storage).
Counters	TAGE counter stored as unsigned 0–7 (taken if ≥ 4), equivalent to a signed 3-bit counter.	None.
Allocation	Deterministic: up to 2 non-consecutive entries; no random table choice. Reproducible runs.	Slightly different allocation distribution from the paper.
SC	Reduced: no local-history, IMLI or return-stack components; fixed training threshold 8 (no dynamic threshold); 7-bit <i>tags</i> added to every SC table, because the assignment requires tag-checked direct-indexed tables (the paper’s SC is untagged). SC may only override a low-confidence TAGE prediction.	SC gives modest gains (see Sec. 4); tags add 7 bits per entry. The invert threshold (4) and TAGE bias weight (12) are compile-time constants.
Loop predictor	16-entry <i>direct-mapped</i> (assignment rule) instead of the paper’s 4-way associative table; no age counter; iteration counter updated at resolve only (no speculative iteration tracking). Tag conflict simply replaces the entry. A wrong confident prediction resets confidence at once (as in Seznec’s design).	Loops with many iterations in flight may be mispredicted near the exit; conflicting loops thrash.
BTB	512 entries with a full 32-bit PC tag and 32-bit target; allocated only at resolve, so a branch’s first execution misses. A BTB miss predicts $PC + 4$ without using TAGE, but TAGE/SC/loop are still trained at resolve. A <code>target_valid</code> bit prevents predicting taken to a never-learned or inherited target.	With full tags, aliasing-induced false branches cannot occur in a 4 KB image; we test those paths with deliberately small BTBs (compile-time <code>PIPESIM.BTB.ENTRIES/TAG_BITS</code>). The BTB is the largest storage item.
Predictor timing	Prediction is synchronous in fetch with zero extra cycles, as specified.	Cycle counts include only stalls and recovery.
Shadow predictors	All three slots are called every fetch/resolve; only slot 0 selects PC. Slot 1/2 are unchanged.	None on slot 0.

4 Correctness Tests, Cycle Counts and Prediction Statistics

4.1 Method and reproduction commands

Correctness is judged by comparing the *final architectural state* (registers `x0--x7`, every non-zero memory word, and the halt status) with a trusted execution, never by the counters alone. The trusted execution is (a) the independent non-speculative interpreter inside `tests/pa3/verify.py` for our own programs, and (b) the supplied `N.expected.txt` files for the ten released tests. Memory is read through `Machine::PrintState`, which already overlays dirty cache lines. The counters are used afterwards to check event accounting.

```
make                                # build ./pipesim
sh tests/run_release_tests.sh      # 10 released tests, own .json each
python3 tests/pa3/verify.py        # 9 programs x 2 configs vs reference
sh tests/pa3/run_alias_stress.sh   # small aliased BTBs
sh tests/pa3/run_predictor_tests.sh # TAGE / loop / SC unit tests
sh tests/run_tests.sh              # PA2 unit + integration tests
./pipesim <prog>.pisa <config>.json 0 # then: n 5000000 / p / s / q
```

4.2 Correctness results

Table 4: All correctness suites pass.

Suite	What is compared	Result
Released tests	final registers, memory, status vs. <code>N.expected.txt</code>	10 / 10
<code>verify.py</code>	registers, memory, halt, retired count, branch counts vs. reference interpreter; 9 programs $\times$ 2 configs	18 / 18
Alias stress	same checks, with 3 deliberately small / partial-tag BTBs	18 / 18 each
Predictor unit tests	TAGE vs. bimodal, loop (both polarities), SC training and reset	all pass
Starter unit + integration	PA2 behaviour retained	pass

Five key test cases. Each is a `.pisa` program with a `.json` configuration and an expected final state produced by the reference interpreter; all five match exactly. Ret. = retired instructions; Evid. = direction errors / squashed completions / stale instruction-fetch replies discarded, showing the intended mechanism ran.

Table 5: Five key correctness cases.

Program	Config	Requirement exercised	Expected final state	Ret.	Evid.
<code>wrongpath_halt</code>	default	Wrong-path HALT, ALU op and store behind a cold taken branch	<code>x1=1, x2=1, x3=5;</code> <code>mem[0x04]=5</code>	6	1 / 0 / 7
<code>wrongpath_late</code>	multicycle	Younger ALU ops finish before a late-resolving branch; late fetch replies	<code>x3=96, x5=3;</code> <code>mem[0x0c]=96</code>	189	9 / 8 / 65
<code>nested_changing</code>	multicycle	Inner loop trip count changes every outer iteration (1..9)	<code>x1=9, x2=9, x3=45,</code> <code>x5=9; mem[0x08]=45</code>	167	13 / 8 / 93
<code>alt_pattern</code>	default	Alternating taken / not-taken branch (direction errors, TAGE learning)	<code>x3=30, x4=1;</code> <code>mem[0x00]=30</code>	306	35 / 0 / 239
<code>btb_alias</code>	multicycle	Two branches sharing one BTB index	<code>x3=30, x4=60;</code> <code>mem[0x10]=60</code>	215	35 / 2 / 22

Registers not listed are 0. In `wrongpath_halt` the program reaches its real HALT after 6 retired instructions and the wrong-path store never reaches memory. With the multicycle configuration the same program also discards one completed wrong-path operation (1 squashed completion).

Target errors and false branch identification. These two events are exercised by rebuilding the simulator with small partial-tag BTBs (`run_alias_stress.sh`). The recovery paths are used heavily and every program still ends in the reference state.

Table 6: Alias stress: events totalled over the 9 programs (default config).

BTB entries	Tag bits	False-branch redirects	Target errors	Programs correct
4	1	8,031	88	18 / 18
8	2	30	58	18 / 18
2	0	50,917	155	18 / 18

Event-accounting checks. On all 18 runs: `branches_resolved` = `correct` + `direction` + `target`; = `conditional` + `unconditional`; = `taken` + `not_taken`; `redirects` = `direction` + `target` + `false_branch_redirects`; each predictor’s `lookups` = `found` + `not_found` and `updates` = `resolved branches`; `squashed_data_requests` = 0; the TAGE provider counters sum to the resolved conditional branches. Stale instruction-fetch replies are discarded in every run (7 to 15,516), and the maximum speculation depth is 1.

4.3 Cycle counts and prediction statistics

Cycles are the simulator’s cycle count (stalls and recovery included; no predictor latency is added). Target errors and false-branch identifications are 0 for all rows in the normal build. $MPKI = 1000 (M_{dir} + M_{tgt}) / I_{retired}$.

Table 7: Measured results, predictor slot 0. *Br* = resolved branches, *Dir* = direction errors.

Program	Retired	Br	default_config				multicycle_config			
			Cycles	IPC	MPKI	Dir	Cycles	IPC	MPKI	Dir
<code>alt_pattern</code>	306	150	774	0.395	114.4	35	1128	0.271	114.4	35
<code>array_sum</code>	182	48	558	0.326	33.0	6	726	0.251	33.0	6
<code>btb_alias</code>	215	120	383	0.561	288.4	62	622	0.346	162.8	35
<code>jump_chain</code>	15	6	103	0.146	200.0	3	112	0.134	200.0	3
<code>long_loop</code>	922	306	1347	0.684	7.6	7	2289	0.403	7.6	7
<code>nested_changing</code>	167	54	384	0.435	77.8	13	555	0.301	77.8	13
<code>sc_random_bias</code>	66059	16000	131843	0.501	33.6	2217	253982	0.260	33.5	2210
<code>wrongpath_halt</code>	6	1	84	0.071	166.7	1	89	0.067	166.7	1
<code>wrongpath_late</code>	189	56	414	0.457	47.6	9	604	0.313	47.6	9

4.4 Component-level behaviour

All counts are for resolved conditional branches over the nine programs with the default configuration (16670 branches). “Provided” is the table whose counter supplied TAGE’s direction before the SC and loop predictor; “correct” means it matched the real outcome.

Table 8: How often each part of TAGE provides the prediction.

Provider	Times provided	Correct	Accuracy
T0	601	443	73.7%
T1	3810	2842	74.6%
T2	2041	1500	73.5%
T3	624	446	71.5%
T4	112	80	71.4%
T5	7	2	28.6%
T6	14	8	57.1%
T7	1	0	0.0%
T8	0	0	–
T9	0	0	–
Base (bimodal)	8400	8347	99.4%
Alternate (weak provider)	1060	724	68.3%

Table 9: Statistical corrector: overrides of TAGE that were later resolved.

Programs	Overrides	Correct	Wrong	Override accuracy
All nine programs	87	59	28	67.8%
Eight small programs	9	3	6	33.3%
<code>sc_random_bias</code>	78	56	22	71.8%

An override is “correct” when TAGE was wrong and the SC fixed it.

Table 10: Loop predictor: resolved branches for which the loop prediction was selected.

Programs	Selected	Correct	Accuracy
All nine programs	421	277	65.8%
<code>long_loop</code> (trip count 50)	99	97	98.0%
<code>nested_changing</code>	14	9	64.3%
<code>sc_random_bias</code> (no real loop)	279	155	55.6%

4.5 Explanation of results

- **Warm-up dominates the small programs.** The direction errors in the first eight programs are mostly first encounters: the BTB is allocated only when a branch resolves, so each static branch misses once, and TAGE needs a few mispredictions to allocate tagged entries. This is why MPKI is high (e.g. 114 for `alt_pattern`, 78 for `nested_changing`) while the long loop reaches 7.6 and the 16,000-branch random-bias program 33.6.
- **Configuration changes cycles, not accuracy.** The multicycle configuration has longer unit latencies and a smaller MSHR/memory-queue setup, so IPC is lower (e.g. 0.684 $\rightarrow$ 0.403 for `long_loop`) while the number of direction errors is largely unchanged; each error costs more cycles because the branch resolves later.
- **The base predictor provides about half of the predictions** (8,400 of 16,670, 99.4% accurate): most branches in these programs are strongly biased. Tables T0–T4 supply most of the rest at 71–75% accuracy; T8 and T9 (history 226 and 359) are never needed because the programs are short.
- **SC gives a small gain.** Over all programs it fixed 59 and broke 28 predictions, a net gain of 31 of the 2,353 direction errors (1.3%). It is active mainly on the random-bias program (56

correct, 22 wrong); on the eight small programs it overrides only 9 times, 3 of them correctly, because it needs training before it deviates from TAGE.

- **The loop predictor is accurate on real loops** (97 of 99 on `long_loop`) and less reliable when the pattern is irregular (155 of 279 on the random-bias branch), where its confidence is repeatedly reset by the immediate-reset rule.
- **Recovery works.** Wrong-path work was discarded (e.g. 8 squashed completions in `wrongpath_late`) and late fetch replies were dropped, with no wrong-path data request ever reaching the cache.

5 Storage Calculations, Access Counts and Trade-offs

5.1 Provisioned logical storage (reported by `StorageBits()`)

Table 11: Logical storage of slot-0 predictor structures.

Structure	Entries	Bits per entry (fields)	Total bits
TAGE base predictor	2048	2 (counter)	4,096
TAGE T0	256	13 (3 ctr + 2 u + 7 tag + 1 valid)	3,328
TAGE T1	256	13	3,328
TAGE T2	256	13	3,328
TAGE T3	256	14 (8-bit tag)	3,584
TAGE T4	128	15 (9-bit tag)	1,920
TAGE T5	256	16 (10-bit tag)	4,096
TAGE T6	128	16 (10-bit tag)	2,048
TAGE T7	64	17 (11-bit tag)	1,088
TAGE T8	64	19 (13-bit tag)	1,216
TAGE T9	64	19 (13-bit tag)	1,216
<code>use_alt_on_na</code>	32	4	128
tick counter	1	8	8
GHR (shared)	1	359	359
PHR (shared)	1	27	27
TAGE subtotal			29,770
SC bias tables (2)	128 + 128	13 (6 ctr + 7 tag)	3,328
SC GEHL tables (4)	64+64+128+128	13	5,056
SC subtotal	640	13	8,320
Loop subtotal	16	62 (26 tag + 16 cur + 16 trip + 2 conf + 1 dir + 1 valid)	992
BTB	512	67 (32 tag + 32 target + 1 valid + 1 cond. + 1 target-valid)	34,304
Total predictor state			73,386

The total, $29,770 + 8,320 + 992 + 34,304 = 73,386$ bits (≈ 9.2 KB), matches the simulator's `predictor0.storage_bits`. Shared state (GHR, PHR) is counted once.

Item	Entries	Bits/entry	Total bits
Prediction record carried per in-flight instruction (slot 0): found, taken, confidence, 32-bit target, 47-bit information	per instr.	82	–
History snapshot ring (GHR 359 + PHR 27)	256	386	98,816
SC lookup snapshots (40 index bits, 42 tag bits, 9-bit score, 2 flags)	256	93	23,808
Branch checkpoints (64-bit sequence + 32-bit predicted next PC)*	128	96	12,288

* A checkpoint also holds a copy of the prediction record above.

Retained metadata and checkpoints (reported separately, not in the total above).

Field widths vs. enforced ranges. TAGE counter: 3 bits, range 0–7 (clamped at 7). *u*: 2 bits, 0–3. `use_alt_on_na`: 4 bits, 0–15. Base counter: 2 bits, 0–3. SC counter: 6 bits, [–32, 31]. SC tag: 7 bits (value 0 remapped to 1). Loop counts: 16 bits, saturate at 65535; confidence 2 bits, max 3. `tick`: 8-bit, wraps at 256. Note: in the **information** word, table-index fields are given 10 bits though 8 would suffice; this affects only metadata, not table storage.

5.2 Component access counts

Per-event table accesses implied by the implementation (maximum / typical case):

Component	Per prediction (Predict)	Per update (resolved branch)
BTB	1 read	1 write
TAGE base	1 read	1 write when no provider, or when provider is weak and no alternate
TAGE tagged	up to 10 reads (scan from T9 down; stops at second match)	1 write for the provider; 1 write to alternate if provider weak; on misprediction up to 10 reads of <i>u</i> and up to 2 allocation writes (or up to 10 <i>u</i> decrements if none free)
<code>use_alt_on_na</code>	1 read (if a provider hit)	1 write when provider is weak and provider/alternate disagree in correctness
SC	6 reads (all tables, every lookup)	Only if training condition holds: 6 reads + 6 writes
Loop	1 read (conditional BTB hit only)	1 read–modify–write per conditional branch

Table 12: Measured table accesses, nine programs, default configuration (86,453 predictions, 16,741 resolved-branch updates, 20,922 SC lookups).

Structure	Access	Total	Average
BTB	table reads	103,194	1.00 per lookup or update
BTB	table writes	16,741	1.00 per update
TAGE base	reads	86,453	1.00 per lookup
TAGE base	writes	8,453	0.50 per update
TAGE tagged	reads	866,728	10.03 per lookup
TAGE tagged	writes	13,853	0.83 per update
<code>use_alt_on_na</code>	reads	10,630	0.12 per lookup
<code>use_alt_on_na</code>	writes	586	0.04 per update
SC bias tables	reads	50,270	2.40 per SC lookup
SC bias tables	writes	8,426	0.50 per update
SC GEHL tables	reads	100,540	4.81 per SC lookup
SC GEHL tables	writes	16,852	1.01 per update
Loop	reads	20,922	1.00 per SC lookup
Loop	writes	16,670	1.00 per update

Reads in the tagged, SC and loop rows include the reads done during update (allocation checks, training). Lookups count every prediction made at fetch, including wrong-path ones; updates count resolved branches only.

5.3 Trade-offs for a lightweight design

- **Small tables** (64–256 entries) and a 3-bit counter / 2-bit u keep TAGE under 30 Kbit; the longest-history tables are the smallest, since few branches need them.
- **Single, reduced SC** (6 tables, 8.3 Kbit, fixed threshold) instead of the full 46.8 Kbit corrector; it only overrides a low-confidence TAGE prediction.
- **16-entry direct-mapped loop predictor** (992 bits): loops are few in the target programs and each entry is 62 bits.
- **History updated at resolve** removes any speculative history repair hardware.
- **Largest cost is the BTB** (47% of storage) because of full 32-bit tags and targets. Possible future saving: partial tags and storing target offsets, at the price of aliasing and false branch identification.

Host runtime and memory use are not part of simulated cycles or logical storage, and these numbers are not area, power or timing estimates.